Java Abstraction — Examples from Easy to Intermediate

A practice PDF covering all abstraction examples provided so far, excluding Animal and Car examples.

Core idea: Abstraction means hiding implementation details and showing only the essential functionality.

1. Easy — Bank Account (Abstract Class)

```
abstract class BankAccount {

    abstract void withdraw();

    void showBankName() {
        System.out.println("ABC Bank");
    }
}

class SavingsAccount extends BankAccount {

    void withdraw() {
        System.out.println("Withdraw from Savings Account");
    }
}

public class Main {
    public static void main(String[] args) {

        SavingsAccount account = new SavingsAccount();

        account.withdraw();
        account.showBankName();
    }
}
```

Output:
Output:
Withdraw from Savings Account
ABC Bank

What to learn: abstract void withdraw() is left without a body. The child class provides the implementation. showBankName() is a normal method shared by the child.

2. Easy — Notification System

```
abstract class Notification {

    abstract void send();

    void showMessage() {
        System.out.println("Sending notification...");
    }
}

class SMSNotification extends Notification {

    void send() {
        System.out.println("Sending SMS");
    }
}

class EmailNotification extends Notification {

    void send() {
        System.out.println("Sending Email");
```

```
    }
}

public class Main {
    public static void main(String[] args) {

        SMSNotification sms = new SMSNotification();
        sms.showMessage();
        sms.send();

        EmailNotification email = new EmailNotification();
        email.showMessage();
        email.send();
    }
}
```

Output:

Output:
Sending notification...
Sending SMS

Sending notification...
Sending Email

What to learn: Every notification must have send(), but each notification type decides how sending works.

3. Easy — Payment Interface

```
interface Payment {

    void pay();
}

class UPI implements Payment {

    public void pay() {
        System.out.println("Payment using UPI");
    }
}

class DebitCard implements Payment {

    public void pay() {
        System.out.println("Payment using Debit Card");
    }
}

public class Main {
    public static void main(String[] args) {

        Payment p = new UPI();
        p.pay();

        p = new DebitCard();
        p.pay();
    }
}
```

Output:

Output:
Payment using UPI
Payment using Debit Card

What to learn: Payment is the contract. UPI and DebitCard provide different implementations.

4. Easy — Login System

```
interface Login {

    void authenticate();
}

class GoogleLogin implements Login {

    public void authenticate() {
        System.out.println("Authenticating using Google");
    }
}

class PasswordLogin implements Login {

    public void authenticate() {
        System.out.println("Authenticating using Password");
    }
}

public class Main {
    public static void main(String[] args) {

        Login login = new GoogleLogin();

        login.authenticate();
    }
}
```

Output:
Output:
Authenticating using Google

What to learn: The interface exposes authenticate(). The internal authentication process is hidden.

5. Easy → Intermediate — Food Delivery

```
interface Delivery {

    void deliver();
}

class BikeDelivery implements Delivery {

    public void deliver() {
        System.out.println("Delivery by bike");
    }
}

class PickupDelivery implements Delivery {

    public void deliver() {
        System.out.println("Customer will pick up the order");
    }
}

public class Main {
    public static void main(String[] args) {

        Delivery d;

        d = new BikeDelivery();
        d.deliver();

        d = new PickupDelivery();
        d.deliver();
    }
}
```

Output:
Output:
Delivery by bike
Customer will pick up the order

What to learn: One Delivery reference can refer to different implementation objects. This demonstrates abstraction plus polymorphism.

6. Intermediate — Employee System

```
abstract class Employee {

    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void showDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }

    abstract void calculateBonus();
}

class Developer extends Employee {

    Developer(String name, double salary) {
        super(name, salary);
    }

    void calculateBonus() {
        System.out.println("Developer bonus: " + salary * 0.10);
    }
}

class Manager extends Employee {

    Manager(String name, double salary) {
        super(name, salary);
    }

    void calculateBonus() {
        System.out.println("Manager bonus: " + salary * 0.20);
    }
}

public class Main {
    public static void main(String[] args) {

        Employee e1 = new Developer("Rahul", 50000);
        Employee e2 = new Manager("Amit", 70000);

        e1.showDetails();
        e1.calculateBonus();

        System.out.println();

        e2.showDetails();
        e2.calculateBonus();
    }
}
```

Output:
Output:

Name: Rahul
Salary: 50000.0
Developer bonus: 5000.0

Name: Amit
Salary: 70000.0
Manager bonus: 14000.0

What to learn: This combines variables, constructor, normal method, abstract method, inheritance, and polymorphism.

7. Intermediate — Report Generator

```
abstract class Report {

    void generateHeader() {
        System.out.println("===== COMPANY REPORT =====");
    }

    abstract void generateBody();

    void generateFooter() {
        System.out.println("===== END OF REPORT =====");
    }
}

class SalesReport extends Report {

    void generateBody() {
        System.out.println("Sales: ■50,000");
    }
}

class EmployeeReport extends Report {

    void generateBody() {
        System.out.println("Employees: 25");
    }
}

public class Main {
    public static void main(String[] args) {

        SalesReport sales = new SalesReport();

        sales.generateHeader();
        sales.generateBody();
        sales.generateFooter();
    }
}
```

Output:
Output:
===== COMPANY REPORT =====
Sales: ■50,000
===== END OF REPORT =====

What to learn: The parent controls the common report structure. The child supplies the report-specific body.

8. Intermediate — Database Connection

```
abstract class Database {

    abstract void connect();
```

```
    void closeConnection() {
        System.out.println("Database connection closed");
    }
}

class MySQLDatabase extends Database {

    void connect() {
        System.out.println("Connected to MySQL");
    }
}

class PostgreSQLDatabase extends Database {

    void connect() {
        System.out.println("Connected to PostgreSQL");
    }
}

public class Main {
    public static void main(String[] args) {

        Database db = new MySQLDatabase();

        db.connect();
        db.closeConnection();
    }
}
```

Output:
Output:
Connected to MySQL
Database connection closed

What to learn: Database defines the common contract and shared closeConnection() behavior. MySQL and PostgreSQL provide their own connection implementation.

9. Intermediate — Multiple Interfaces

```
interface Printable {

    void print();
}

interface Scannable {

    void scan();
}

class Printer implements Printable, Scannable {

    public void print() {
        System.out.println("Printing document");
    }

    public void scan() {
        System.out.println("Scanning document");
    }
}

public class Main {
    public static void main(String[] args) {

        Printer p = new Printer();

        p.print();
        p.scan();
    }
}
```

Output:
Output:
Printing document
Scanning document

What to learn: A class can implement multiple interfaces. Each interface represents a separate capability or contract.

10. Intermediate — Abstract Class + Interface

```
interface Authentication {

    void login();
}

abstract class User implements Authentication {

    String username;

    User(String username) {
        this.username = username;
    }

    void showUsername() {
        System.out.println("Username: " + username);
    }
}

class Admin extends User {

    Admin(String username) {
        super(username);
    }

    public void login() {
        System.out.println("Admin login successful");
    }
}

public class Main {
    public static void main(String[] args) {

        Admin admin = new Admin("Pratham");

        admin.showUsername();
        admin.login();
    }
}
```

Output:
Output:
Username: Pratham
Admin login successful

What to learn: This combines interface, abstract class, constructor, inheritance, and implementation.

11. Advanced Practice — E-Commerce Order

```
interface Payment {

    void pay(double amount);
}

class UPI implements Payment {

    public void pay(double amount) {
        System.out.println("Paid ■" + amount + " using UPI");
```

```
    }
}

class Card implements Payment {

    public void pay(double amount) {
        System.out.println("Paid ■" + amount + " using Card");
    }
}

class Order {

    void checkout(Payment payment, double amount) {

        System.out.println("Processing order...");

        payment.pay(amount);

        System.out.println("Order completed");
    }
}

public class Main {
    public static void main(String[] args) {

        Order order = new Order();

        Payment payment = new UPI();

        order.checkout(payment, 1500);
    }
}
```

Output:

Output:

Processing order...

Paid ■1500.0 using UPI

Order completed

What to learn: Order depends on the Payment abstraction, not on a specific payment implementation. You can replace UPI with Card without changing Order.

Quick Revision — Abstraction Rules

1. Abstraction hides implementation details and exposes essential functionality.
2. Java mainly achieves abstraction using abstract classes and interfaces.
3. An abstract class can contain abstract methods, normal methods, variables, and constructors.
4. An abstract method has no method body and must eventually be implemented by a concrete child class.
5. You cannot directly create an object of an abstract class.
6. An interface cannot be directly instantiated.
7. A class uses extends for a class and implements for an interface.
8. A class can implement multiple interfaces.
9. An interface can extend multiple interfaces.
10. Interface references can point to implementation objects, for example: Payment p = new UPI();
11. Abstract class = common code/state + incomplete behavior.
12. Interface = contract/capability; modern interfaces can also contain default, static, and private methods.

Important Syntax to Memorize

```
// Abstract class
abstract class Example {
    abstract void doSomething();

    void commonMethod() {
        System.out.println("Common");
    }
}

// Interface
interface ExampleInterface {
    void doSomething();
}

// Implementing an interface
class Child implements ExampleInterface {
    public void doSomething() {
        System.out.println("Done");
    }
}

// Extending an abstract/concrete class
class ChildClass extends Example {
    void doSomething() {
        System.out.println("Done");
    }
}

// Polymorphic reference
ExampleInterface obj = new Child();
```